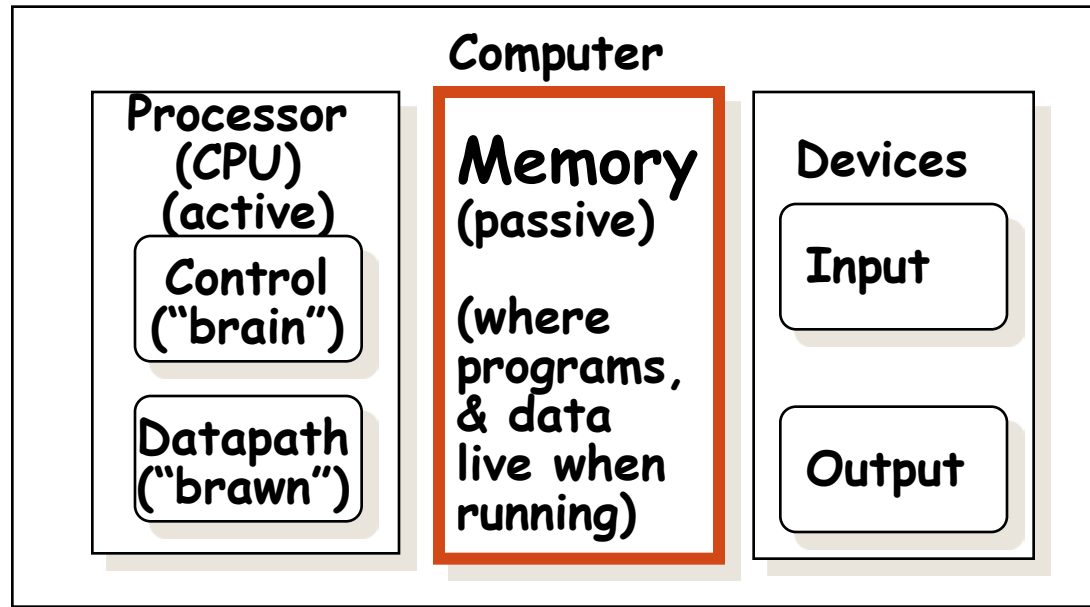
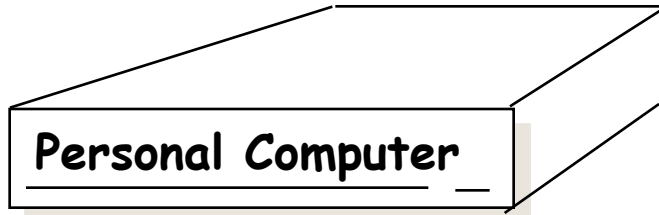


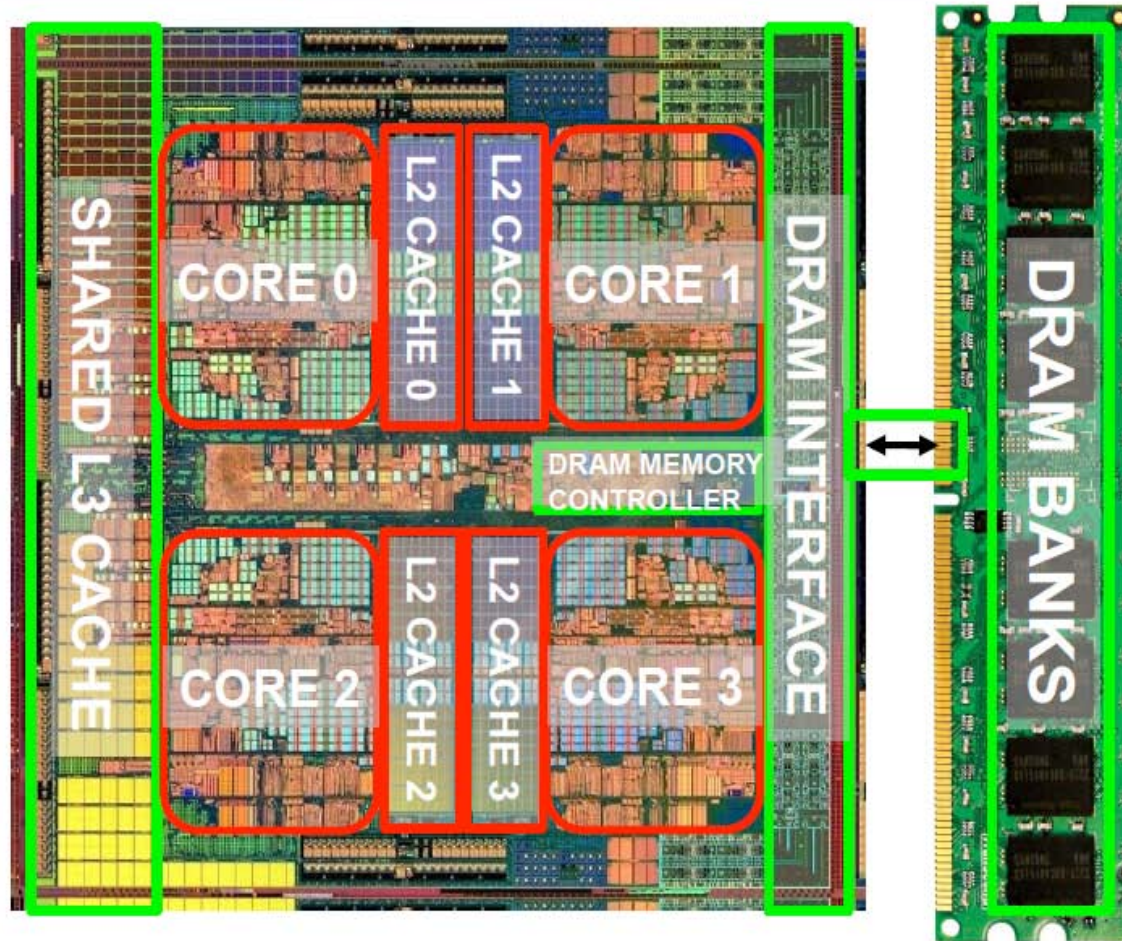
Memory Hierarchy

Cache Management

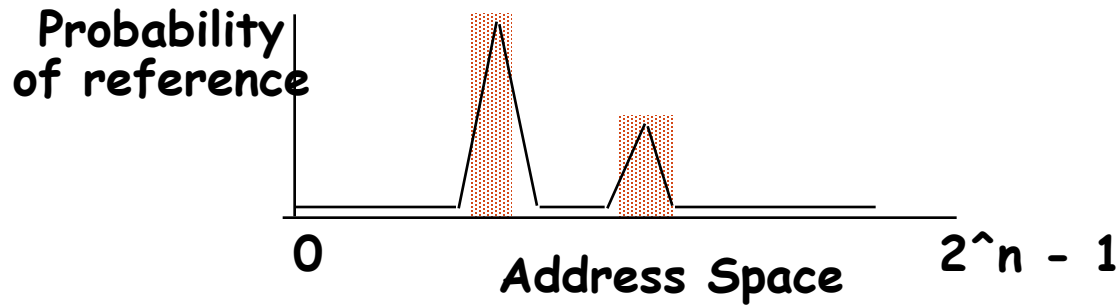
Recap: Computer Organization:



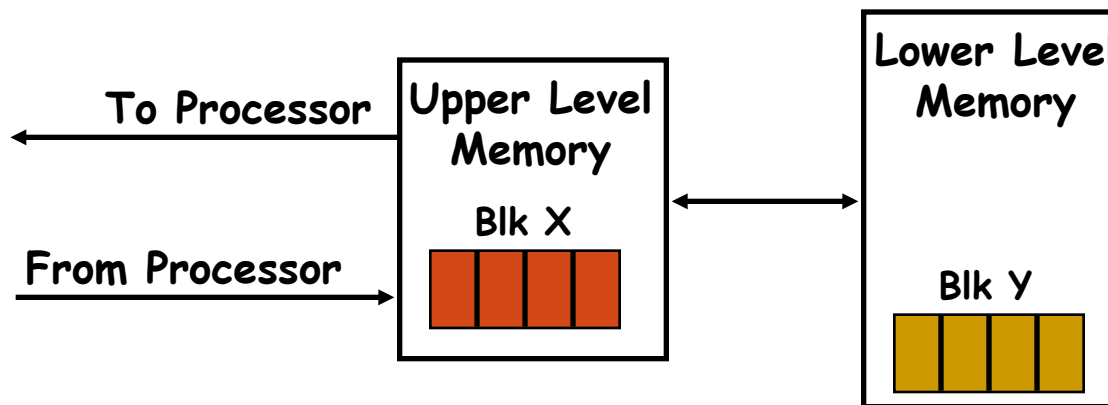
Memory Hierarchy: Cache vs Main Memory



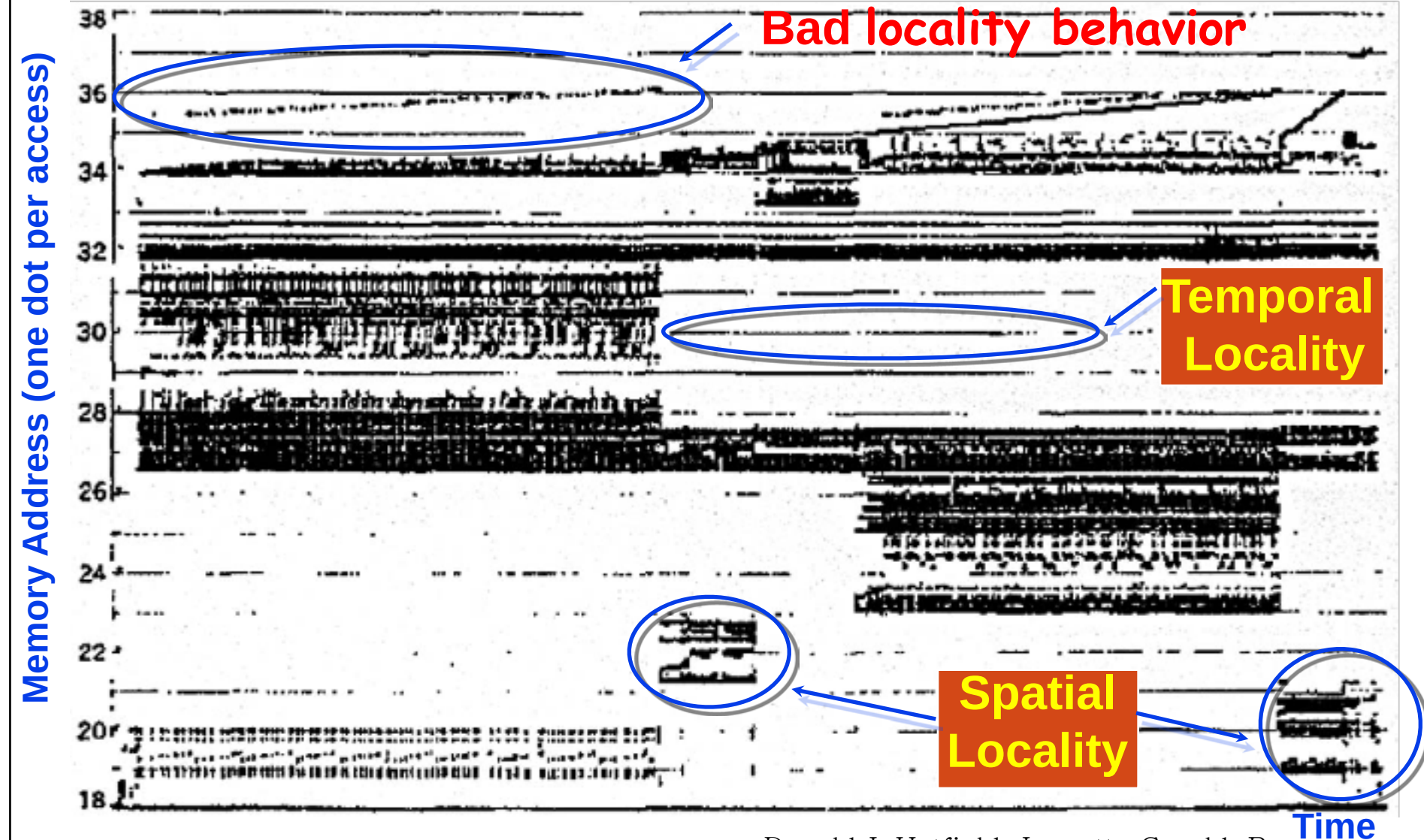
Memory Hierarchy: Why Does it Work? Locality!



- Temporal Locality (Locality in Time):
=> Keep most recently accessed data items closer to the processor
- Spatial Locality (Locality in Space):
=> Move blocks consists of contiguous words to the upper levels



Programs with locality cache well ...



Donald J. Hatfield, Jeanette Gerald: Program
Restructuring for Virtual Memory. IBM Systems
Journal 10(3): 168-192 (1971)

The Goal: Illusion of **large, fast, cheap** memory

- Fact: Large memories are slow, fast memories are small
- How do we create a memory that is large, cheap and fast (most of the time)?
- **Hierarchy of Levels**
 - Uses smaller and faster memory technologies close to the processor
 - Fast access time in highest level of hierarchy
 - Cheap, slow memory furthest from processor
- The **aim of memory hierarchy design** is to have access time close to the highest level and size equal to the lowest level

How is the hierarchy managed?

- Registers $\longleftrightarrow$ Memory Space (any 'virtually')
 - By the compiler (or assembly language Programmer)
- Cache $\longleftrightarrow$ Main Memory
 - By hardware
- Main Memory $\longleftrightarrow$ Disks
 - operating system (virtual memory: pages)
 - Swap space for each 'process'

Introduction to Caches

- Cache
 - is a small very fast memory (SRAM, expensive)
 - contains copies of the most recently accessed memory locations (data and instructions): *temporal locality*
 - is fully *managed by hardware* (unlike virtual memory)
 - storage is organized in *blocks* of contiguous memory locations: *spatial locality*
 - unit of transfer to/from main memory (or L2) is the *cache block*
- *General structure*
 - *n blocks* per cache organized in *s sets*
 - *b bytes* per block (block size)
 - *total cache size $n*b$ bytes*

Cache: Terminology

Hit: data appears in level X: Hit Rate: the fraction of memory accesses found in the upper level

Miss: data needs to be retrieved from a block in the lower level
(Block Y) Miss Rate = $1 - (\text{Hit Rate})$

Hit Time: Time to access the upper level which consists of Time to determine hit/miss + memory access time

Miss Penalty: Time to replace a block in the upper level + Time to deliver the block to the processor

Note: $\text{Hit Time} \ll \text{Miss Penalty}$

Measuring Cache Performance

CPU time = Execution cycles X clock cycle time

If cache miss: (Execution cycles + **Memory stall cycles**) X clock cycle time

Read-stall cycles = **# reads** X Read miss rate X Read miss penalty

Write-stall cycles = **# writes** X write miss rate X write miss penalty

Memory-stall cycles = Read-stall + write-stall

= **Memory accesses** X miss rate X miss penalty

= **# instns** X **misses/instn** X miss penalty

Example

Q: Cache miss penalty = 50 cycles and all instns take 2.0 cycles without memory stalls. Assume cache miss rate of 2% and 1.33 memory references per instn. What is the impact of cache?

Ans: CPU time = $IC \times (CPI + \text{Memory stall cycles/instn}) \times \text{cycle time } t$

Performance including cache misses is

CPU time = $IC \times (2.0 + (1.33 \times .02 \times 50)) \times \text{cycle time} = IC \times 3.33 \times t$

For a perfect cache that never misses CPU time = $IC \times 2.0 \times t$

Hence, including the memory hierarchy stretches CPU time by 1.33

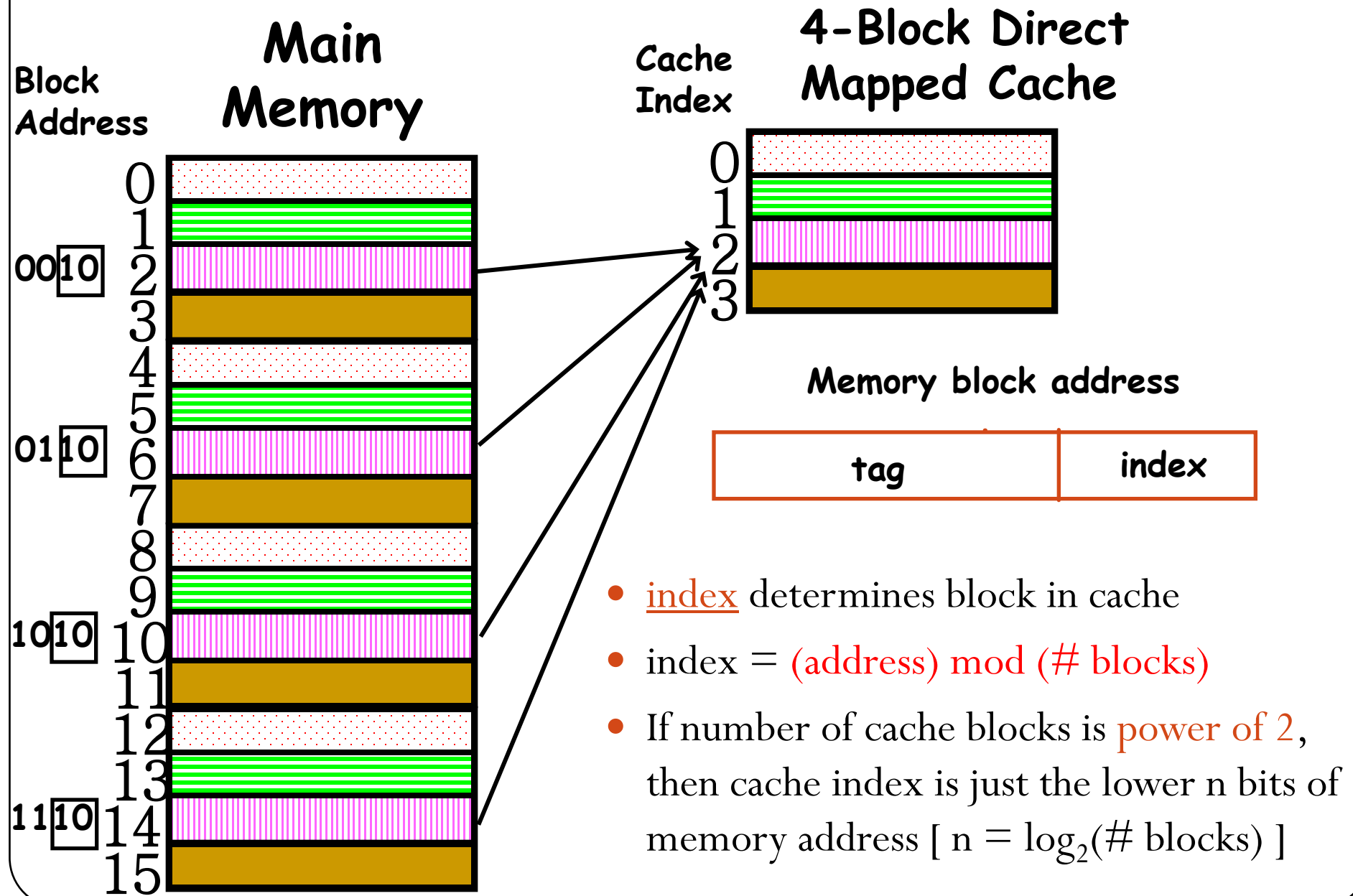
But, without memory hierarchy, the CPI would increase to

$2.0 + 50 \times 1.33 \times 1$ or 68.5 – a factor of over 30 times longer.

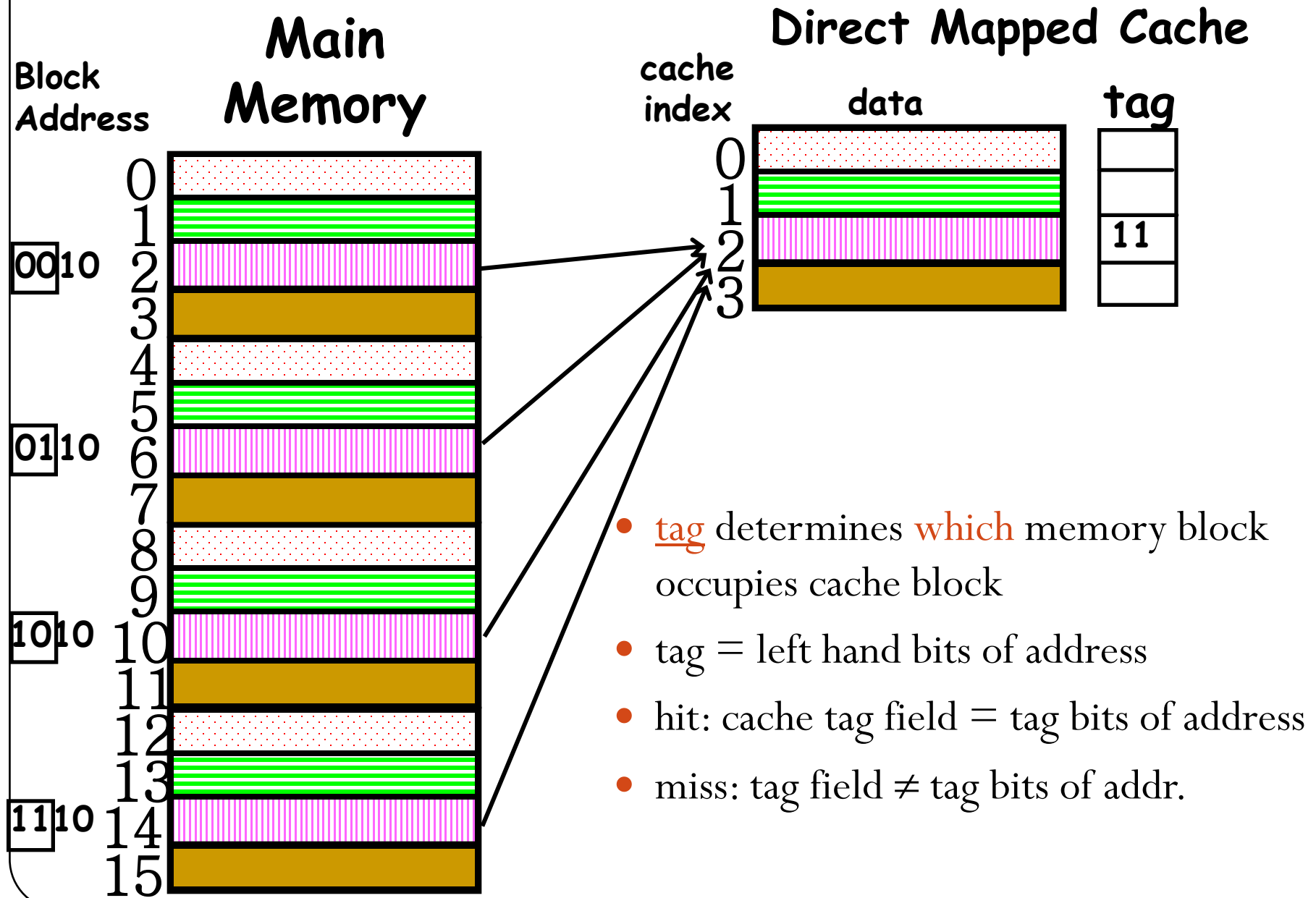
Cache Organization

- (1) How do you know if something is in the cache?
- (2) If it is in the cache, how to find it in block?
- Answer to (1) and (2) depends on type or **organization of the cache**
- In a **direct mapped cache**, each memory address (byte) is associated with one possible block within the cache
 - Therefore, we only need to look in a single location in the cache for the data if it exists in the cache

Simplest Cache: Direct Mapped



Simplest Cache: Direct Mapped

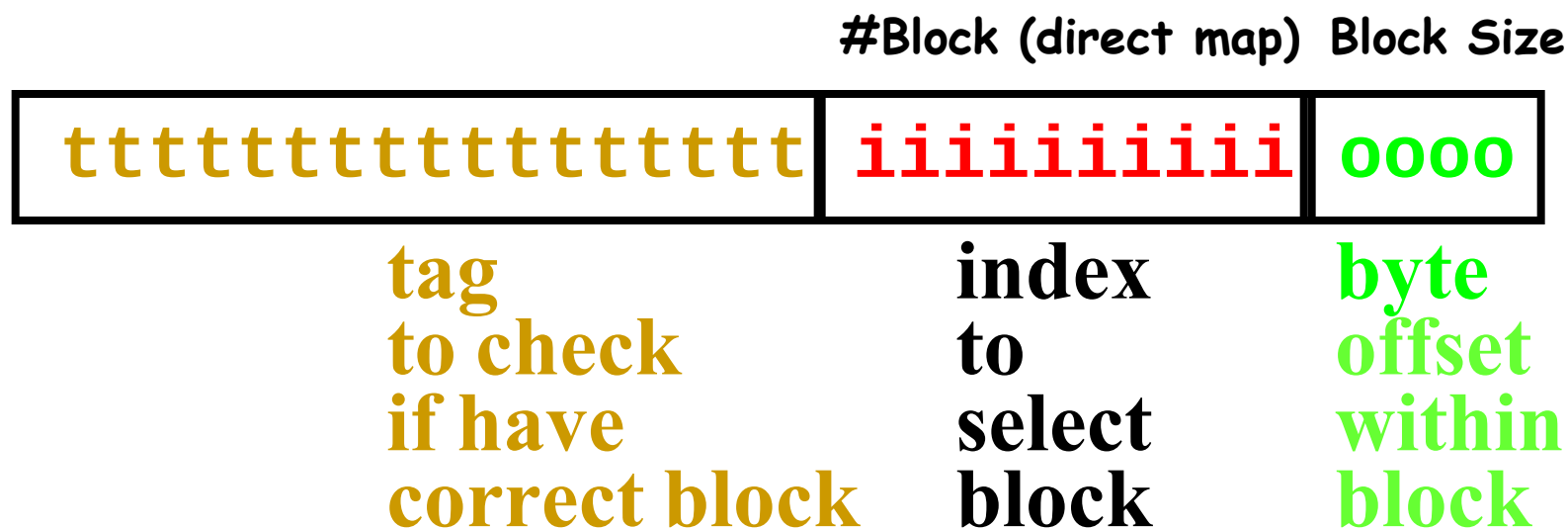


Finding Item within Block

- In reality, a cache block consists of a number of bytes/ words (32 or 64 bytes) to (1) increase cache hit due to locality property and (2) reduce the cache miss time.
- Mapping: memory block I can be mapped to cache block frame $I \bmod x$, where x is the number of blocks in the cache
 - Called congruent mapping
- Given an address of item, **index** tells which block of cache to look in
- Then, how to find requested item **within** the cache block?
- Or, equivalently, “What is the byte offset of the item within the cache block?”

Direct-Mapping

- If **block size** > 1 , rightmost bits of index are really the offset within the indexed block



Accessing data in a direct mapped cache

Three types of events:

- cache miss: nothing in cache in appropriate block, so fetch from memory
- cache hit: cache block is valid and contains proper address, so read desired word
- cache miss, block replacement: wrong data is in cache at appropriate block, so discard it and fetch desired data from memory

Cache Access Procedure: (1) Use **Index** bits to select cache block
(2) If **valid bit** is 1, compare the **tag** bits of the address with the cache block tag bits (3) If they match, use the **offset** to read out the word/byte.

Data valid, tag OK, so read offset return word d

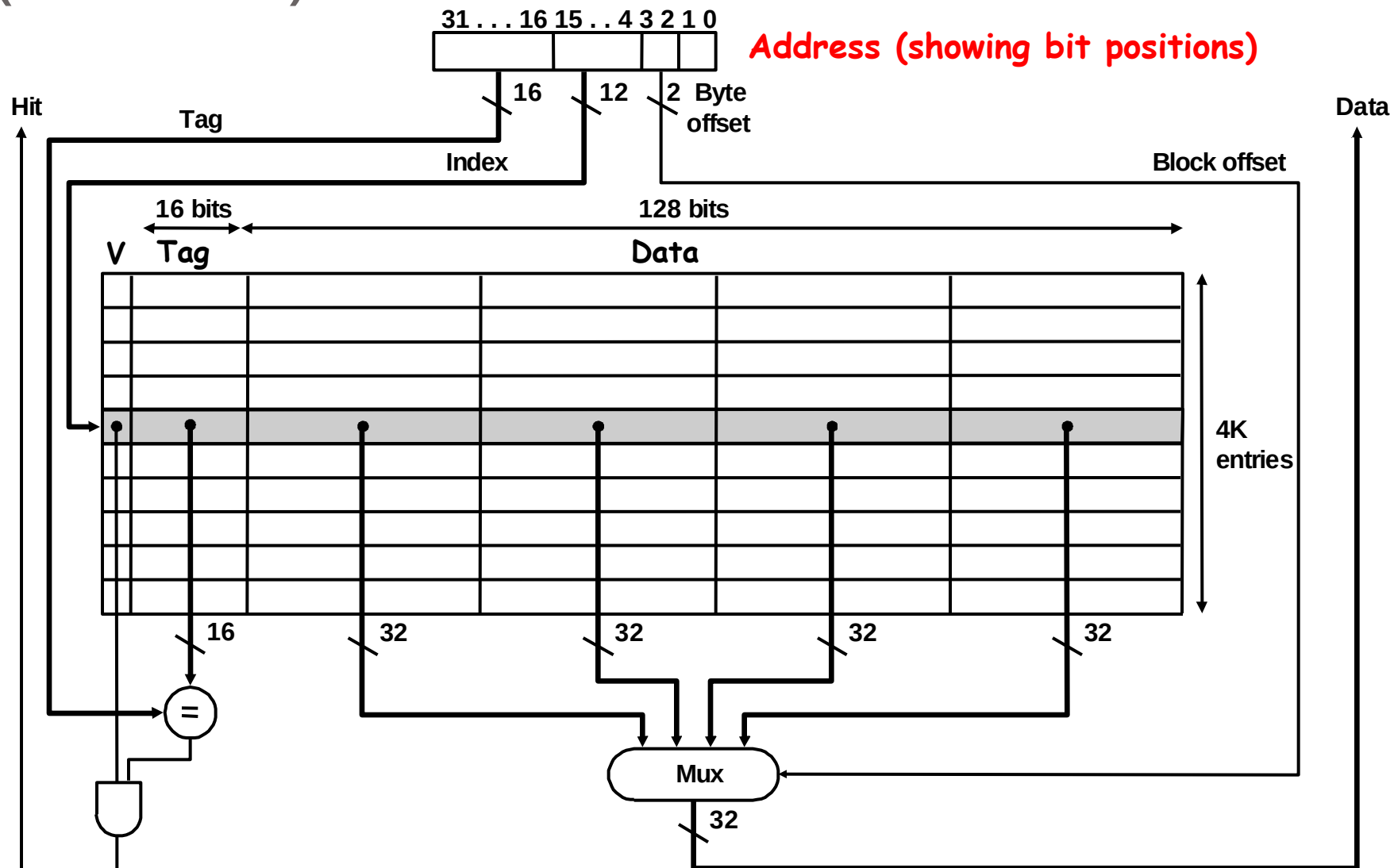
• 000000000000000000000000 000000000001 1100

Index		Tag	1 0x0-3	0x4-7	0x8-b	3 <u>0xc-f</u>
0	0					
<u>1</u>	<u>1</u>	<u>0</u>	a	b	c	d
2	0					
3	0					
4	0					
5	0					
6	0					
7	0					

... Valid ...

1022	0					
1023	0					

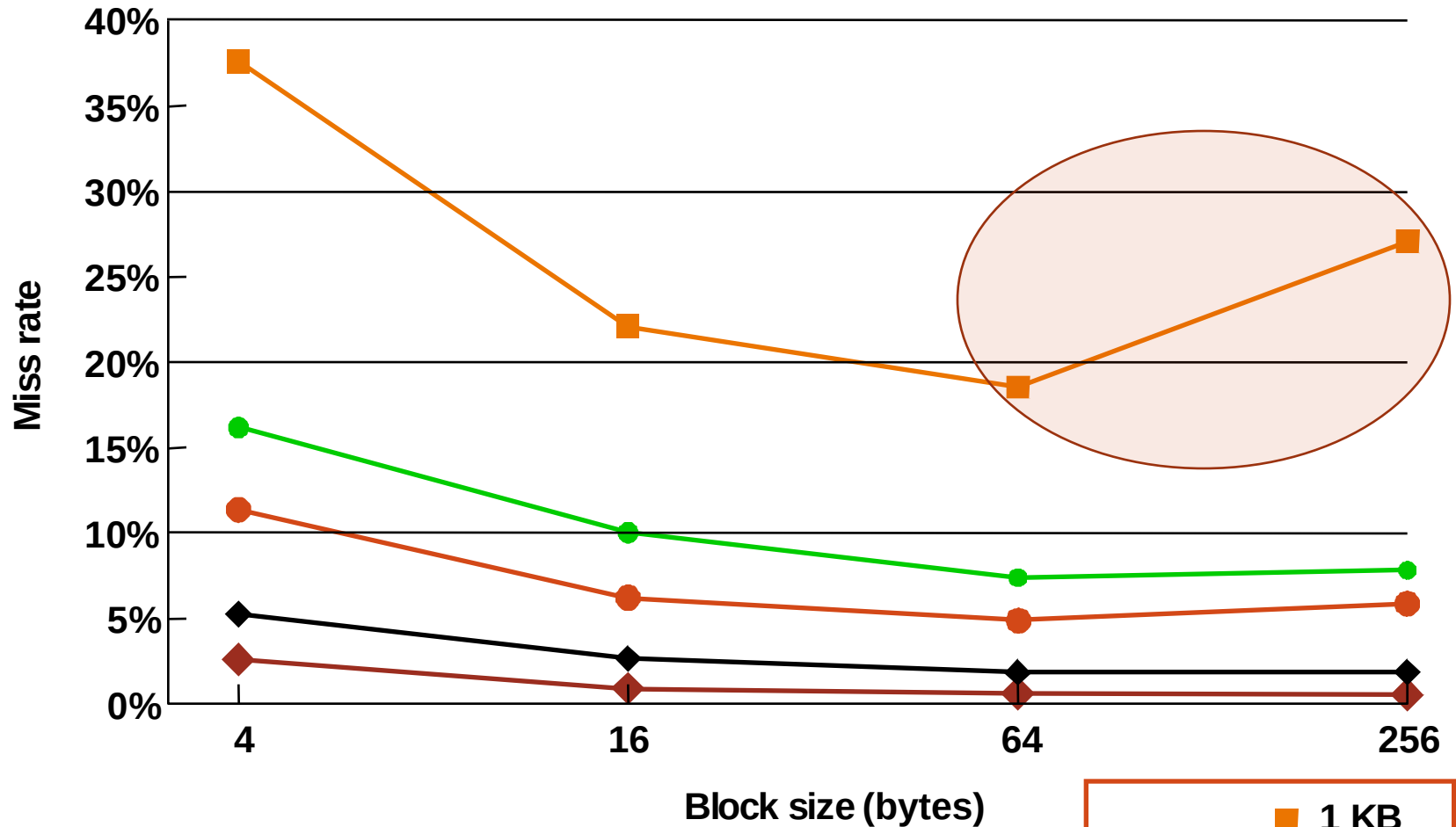
64KB Cache with 4-word (16-byte) blocks (Hardware)



Miss rates: 1-word vs. 4-word **block size**

Program		I-cache miss rate	D-cache miss rate	Combined miss rate
1-word block	gcc	6.1%	2.1%	5.4%
	spice	1.2%	1.3%	1.2%
4-word block	gcc	2.0%	1.7%	1.9%
	spice	0.3%	0.6%	0.4%

Miss Rate Versus Block Size



for direct mapped cache

**total
cache
size**

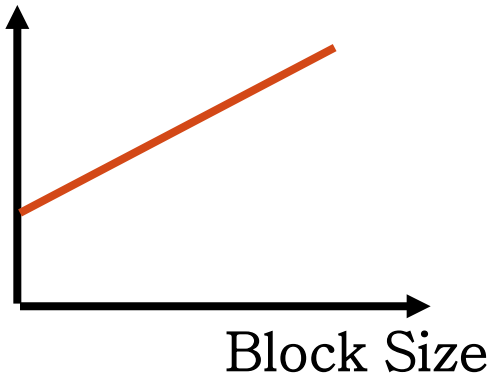
- 1 KB
- 8 KB
- 16 KB
- 64 KB
- 256 KB

Block Size and Miss Penalty

- With increase in **block size**, the **cost of a miss** also increases
- **Miss penalty**: time to fetch the block from the next lower level of the hierarchy and load it into the cache
- With very large blocks, increase in miss penalty overwhelms decrease in miss rate
- Can minimize **average access time** if design memory system right

Block Size Tradeoff

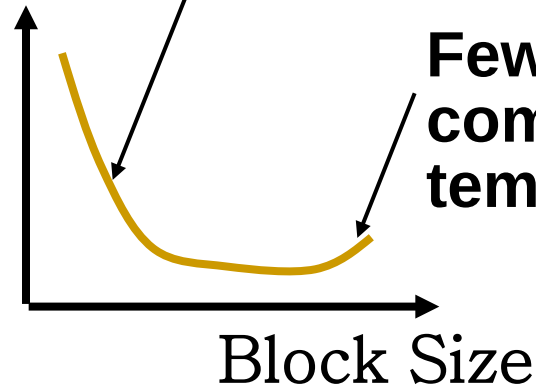
Miss
Penalty



Miss
Rate

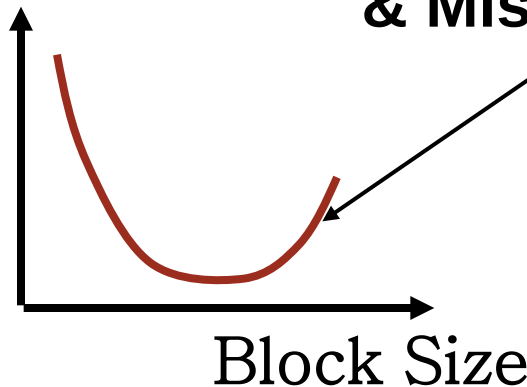
Exploits Spatial Locality

Fewer blocks:
compromises
temporal locality



Average
Access
Time=

Increased Miss Penalty
& Miss Rate



Extreme Example: 1-block **cache size**

- Suppose choose block size = cache size? Then only one block in the cache
- Temporal Locality says if an item is accessed, it is likely to be accessed again soon
 - But it is unlikely that it will be accessed again immediately!!!
 - The next access is likely to be a miss
 - Continually loading data into the cache but forced to discard them before they are used again

Direct-mapped Cache Contd.

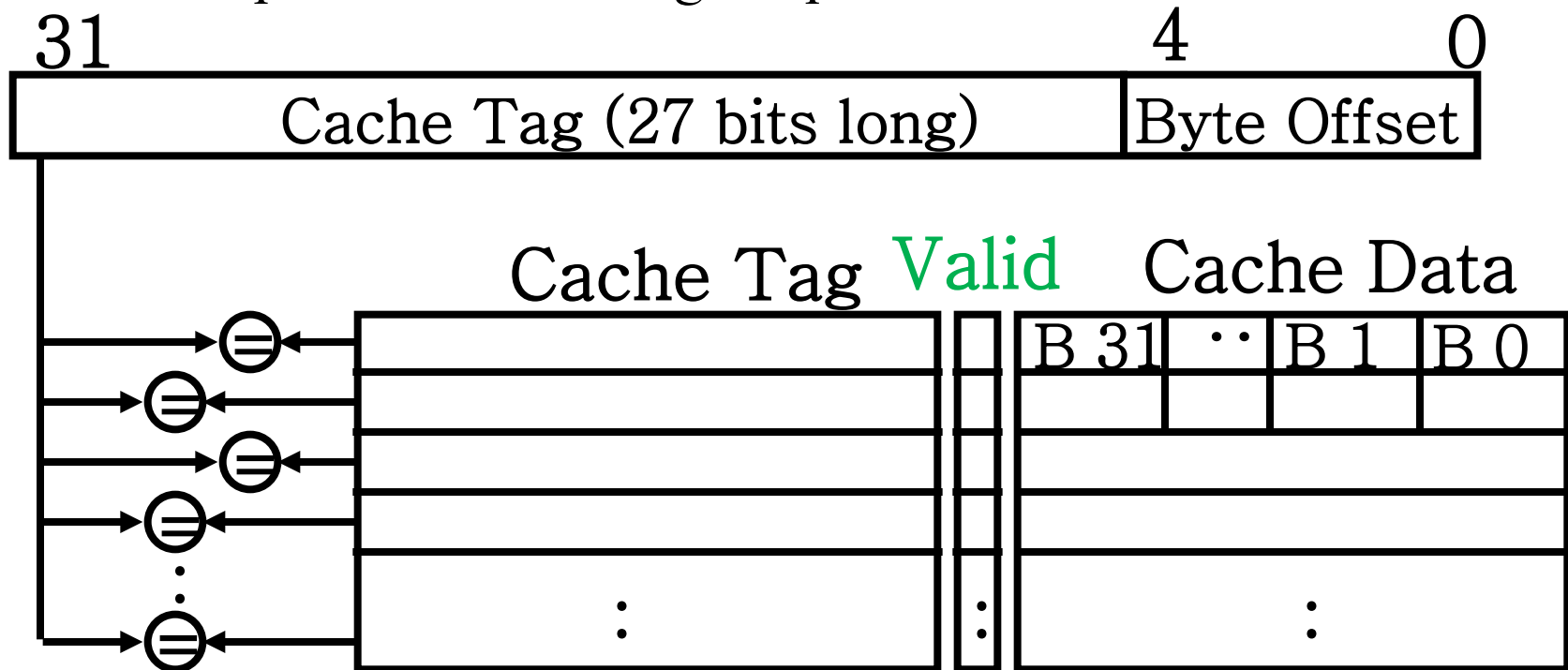
- The direct mapped cache is simple to design and its access time is fast
- Good for L1 (on-chip cache)
- Problem: Conflict Miss, so low hit ratio

Conflict Misses are misses caused by accessing different memory locations that are mapped to the same cache index

In **direct mapped cache**, no flexibility in where memory block can be placed in cache, contributing to conflict misses

Another Extreme: Fully Associative

- Fully Associative Cache (8 word block)
 - Omit cache index; place item in any block!
 - Compare all Cache Tags in parallel



By definition: Conflict Misses = 0 for a fully associative cache

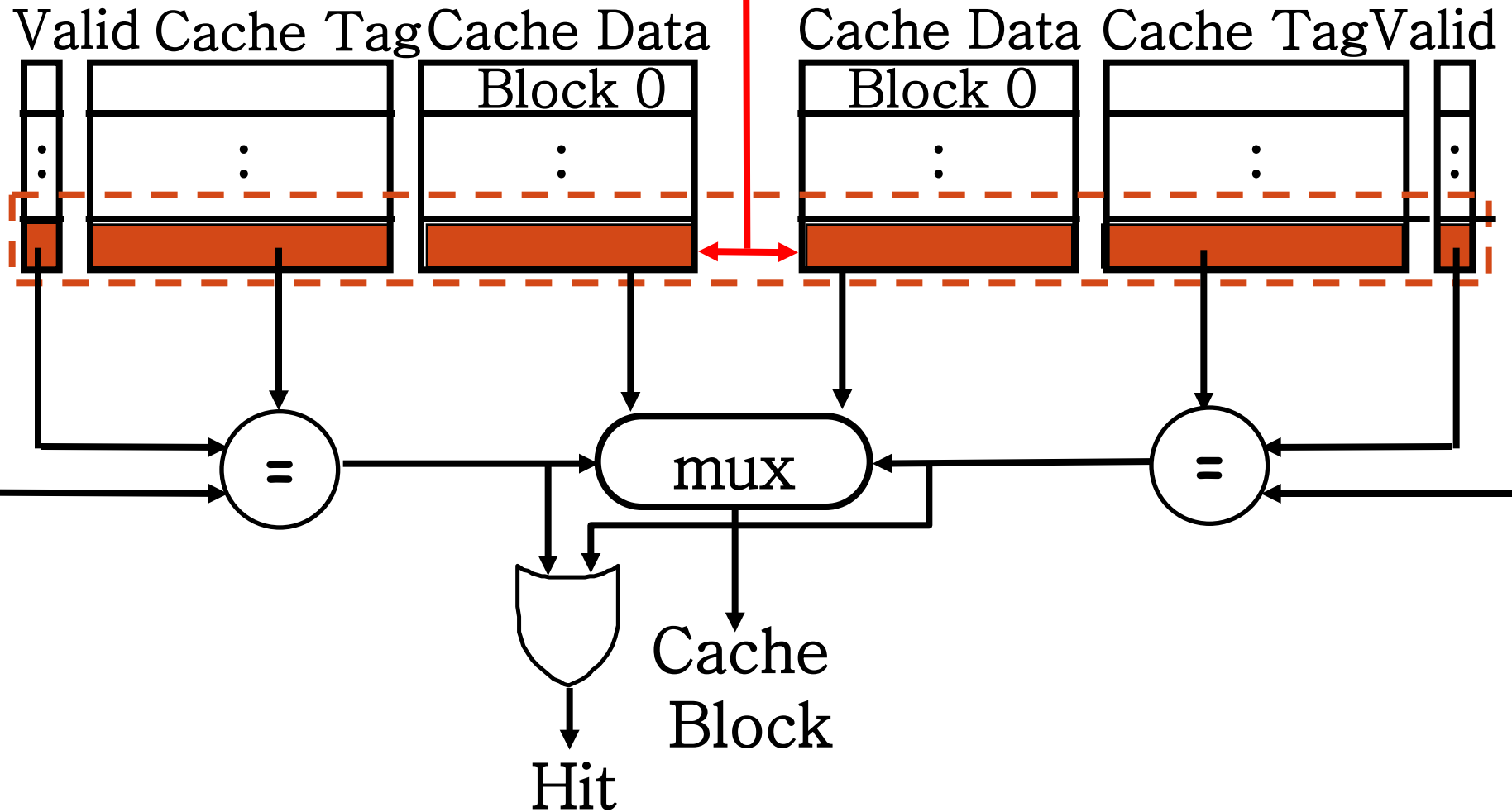
Fully Associative Cache

- Must search all tags in cache, as item can be in any cache block
- Search for tag must be done by hardware in **parallel** (other searches too slow)
- But, the necessary parallel comparator hardware is very expensive
- Therefore, fully associative placement practical only for a very small cache

Compromise: N-way Set Associative Cache

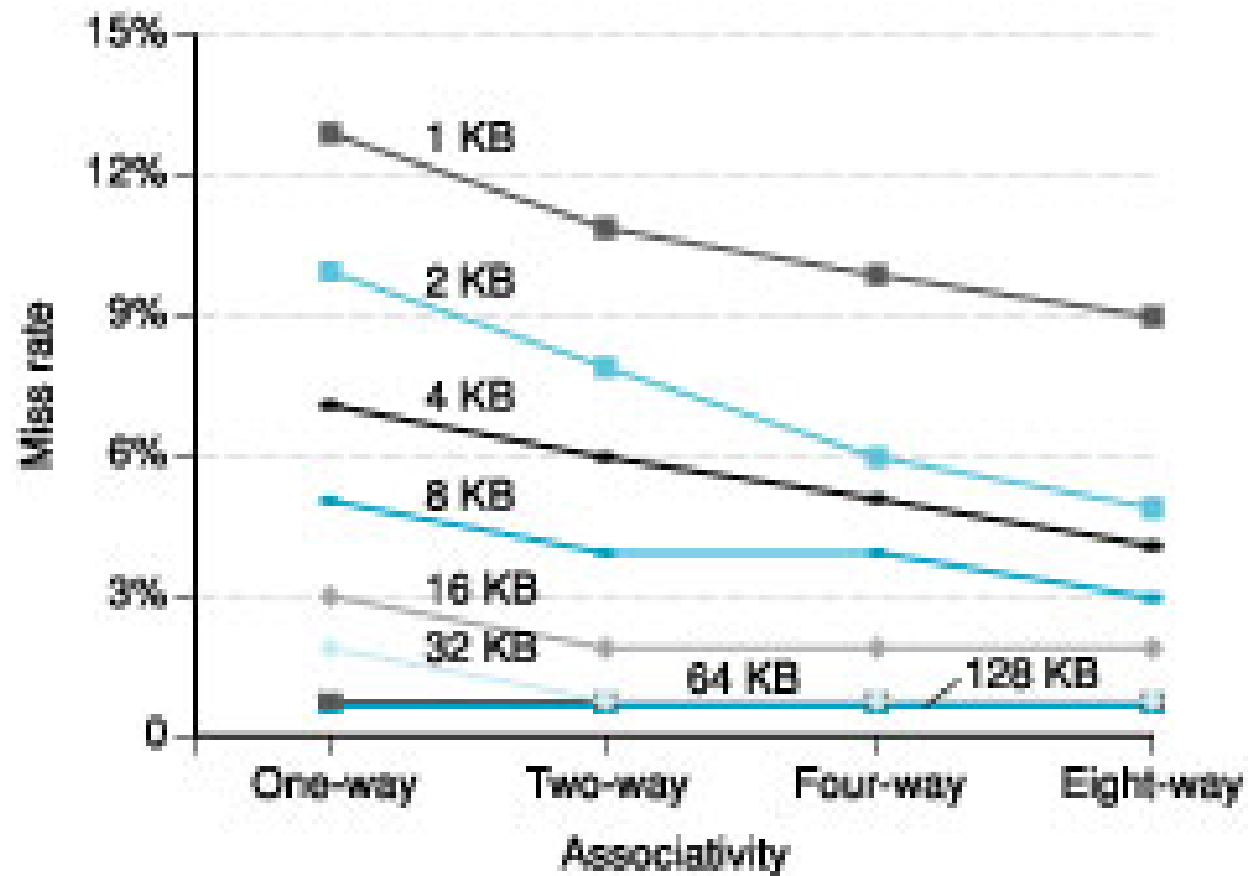
- N-way set associative:
N cache blocks for each Cache Index
 - Like having N direct mapped caches operating in parallel
 - Select the one that gets a hit
- Example: 2-way set associative cache
 - Cache Index selects a “set” of 2 blocks from the cache
 - The 2 tags in set are compared in parallel
 - Data is selected based on the tag result (which matched the address)

Example: 2-way Set Associative Cache



Set Associative Cache Contd.

- Direct Mapped, Fully Associative can be seen as just variations of Set Associative block placement strategy
- Direct Mapped =
 1-way Set Associative Cache
- Fully Associative =
 n-way Set associativity for a cache with exactly n blocks



Block Replacement Policy

- N-way Set Associative or Fully Associative have choice where to place a block, (which block to replace)
 - Of course, if there is an invalid block, use it
- Whenever get a cache hit, record the cache block that was touched
- When need to evict a cache block, choose one which hasn't been touched recently: “Least Recently Used” (LRU)
 - Past is prologue: history suggests it is least likely of the choices to be used soon
 - Opposite of temporal locality

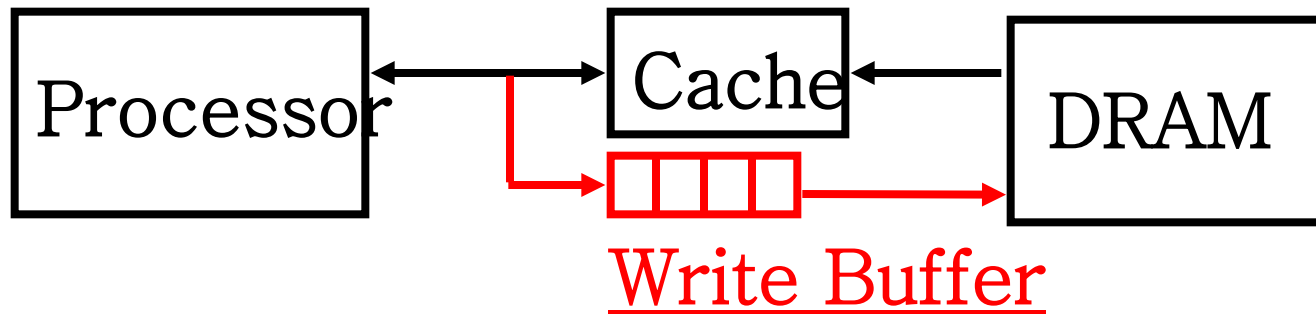
Block Replacement Policy: Random

- Sometimes hard to keep track of the LRU block if lots of choices
- *How hard for 2-way associativity?*
- Second Choice Policy: pick one at random and replace that block
- Advantages
 - Very simple to implement
 - Predictable behavior
 - No worst case behavior

What about Writes to Memory?

- Suppose write data only to cache?
 - Main memory and cache would then be inconsistent - cannot allow
- Simplest Policy: The information is written to both the block in the cache and to the block in the lower-level memory (write-through)
- Problem: Writes operate at speed of lower level memory!

Improving Cache Performance: Write Buffer



- A Write Buffer is added between Cache and Memory
 - Processor: writes data into cache & write buffer
 - Controller: write buffer contents to memory
- Write buffer is just a First-In First-Out queue:
 - Typical number of entries: 4 to 10
 - Works fine if: $\text{Store frequency (w.r.t. time)} \ll 1 / \text{DRAM write cycle time}$

Improving Cache Performance: Write Back

- Option 2: data is written only to cache block
- Modified cache block is written to main memory only when it is replaced
 - Block is unmodified (clean) or modified (dirty)
- This scheme is called “Write Back”
 - Advantage? Repeated writes to same block stay in cache
 - Disadvantage? More complex to implement

Improving Caches Performance

- In general, want to minimize

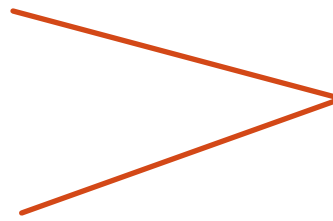
Average Access Time:

$$= \text{Hit Time} \times (1 - \text{Miss Rate}) \\ + \text{Miss Penalty} \times \text{Miss Rate}$$

(recall $\text{Hit Time} \ll \text{Miss Penalty}$)

- So far, have looked at

- Larger Block Size
- Larger Cache
- Higher Associativity



**Reduce
Miss Rate**

- What else to reduce miss penalty? Add a second level (L2) cache.

Multilevel cache: How do we calculate the miss penalty?

- Access time = L1 hit time * L1 hit rate + L1 miss penalty * L1 miss rate
- We simply calculate the L1 miss penalty as being the access time for the L2 cache
- Access time = L1 hit time * L1 hit rate + (L2 hit time * L2 hit rate + L2 miss penalty * L2 miss rate) * L1 miss rate

Do the numbers for L2 Cache

- Assumptions:
 - L1 hit time = 1 cycle, L1 hit rate = 90%
 - L2 hit time (also L1 miss penalty) = 4 cycles,
L2 miss penalty = 100 cycles,
L2 hit rate = 90%
- Access time = L1 hit time * L1 hit rate + (L2 hit time * L2 hit rate +
L2 miss penalty * (1 - L2 hit rate))
* L1 miss rate
$$= 1 * 0.9 + (4 * 0.9 + 100 * 0.1) * (1 - 0.9)$$
$$= 0.9 + (13.6) * 0.1 = 2.26 \text{ clock cycles}$$

What would it be without the L2 cache?

- Assume that the L1 miss penalty would be 100 clock cycles
- $1 * 0.9 + (100) * 0.1$
- 10.9 clock cycles vs. 2.26 with L2
- So gain a benefit from having the second, larger cache before main memory
- Today's L1 cache sizes: 16 KB-64 KB;
L2 cache may be 512 KB to 4096 KB

An Example

Q: Suppose we have a processor with a base CPI of 1.0 assuming all references hit in the primary cache and a clock rate of 500 MHz. The main memory access time is 200 ns. Suppose the miss rate per instn is 5%. What is the revised CPI? How much faster will the machine run if we put a secondary cache (with 20-ns access time) that reduces the miss rate to memory to 2%? Assume same access time for hit or miss.

A: Miss penalty to main memory = 200 ns = 100 cycles. Total CPI = Base CPI + Memory-stall cycles per instn. Hence, revised CPI = $1.0 + 5\% \times 100 = 6.0$

When an L2 with 20-ns (10 cycles) access time is put, the miss rate to memory is reduced to 2%. So, out of 5% L1 miss, L2 hit is 3% and miss is 2%.

The CPI is reduced to $1.0 + 5\% \times (10 + 40\% \times 100) = 3.5$. Thus, the m/c with secondary cache is faster by $6.0/3.5 = 1.7$

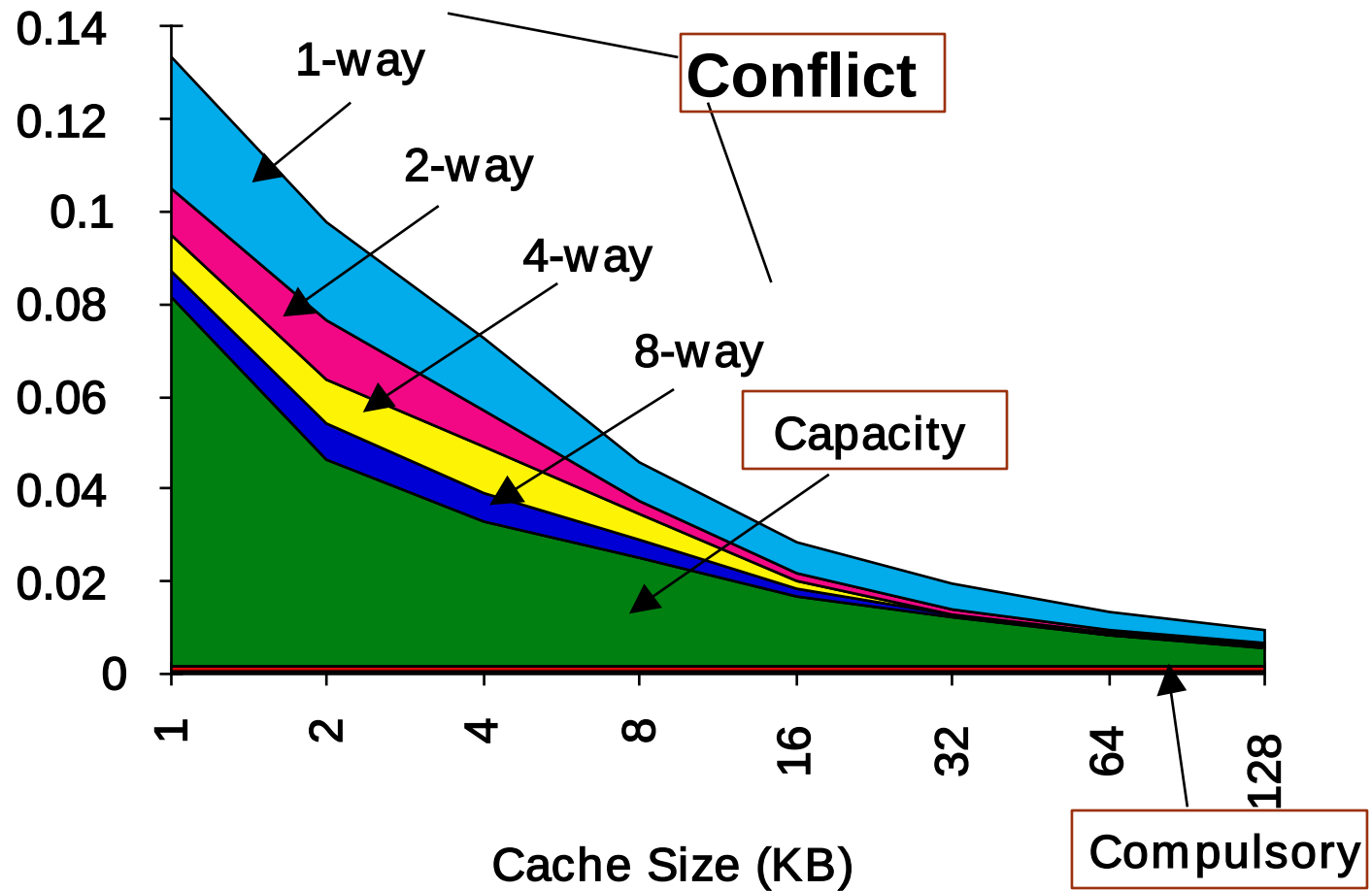
Where to misses come from?

- Classifying Misses: 3 Cs
 - *Compulsory*—The first access to a block is not in the cache, so the block must be brought into the cache. Also called *cold start misses* or *first reference misses*.
(*Misses in even an Infinite Cache*)
 - *Capacity*—If the cache cannot contain all the blocks needed during execution of a program, *capacity misses* will occur due to blocks being discarded and later retrieved.
(*Misses in Fully Associative Size X Cache*)
 - *Conflict*—If block-placement strategy is set associative or direct mapped, conflict misses (in addition to compulsory & capacity misses) will occur because a block can be discarded and later retrieved if too many blocks map to its set. Also called *collision misses* or *interference misses*.
(*Misses in Direct / N-way Associative, Size X Cache*)

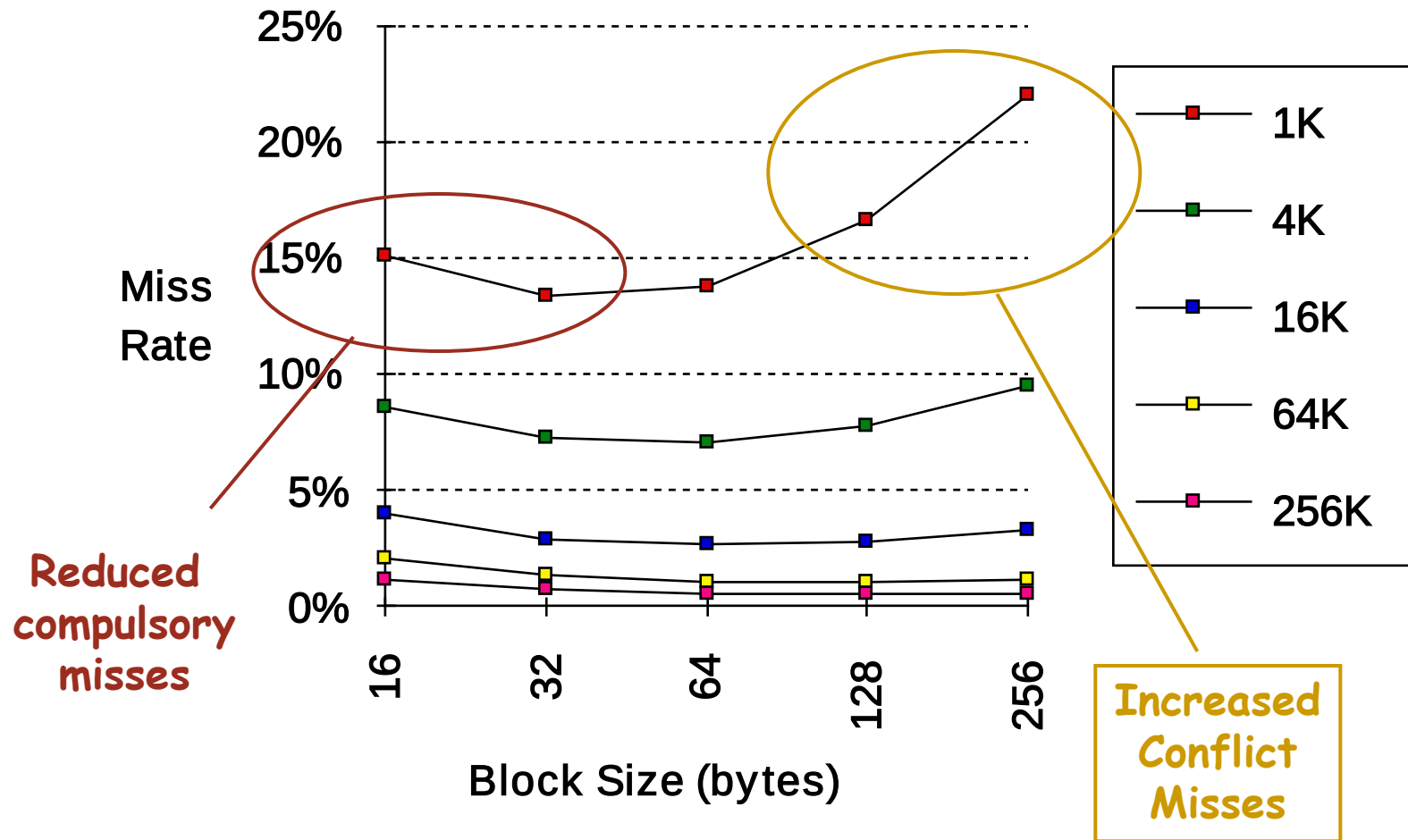
How to Address Three Cs

- The cache miss of three classes.
 - Compulsory misses: Caused due to first access to the block from memory — **small but fixed independent of cache size.**
 - Capacity misses: Because the cache cannot contain all the blocks for its limited size — **reduces by increasing cache size**
 - Conflict misses: Because multiple blocks compete for the same block or set in the cache. Also called collision misses. — **reduces by increasing associativity**

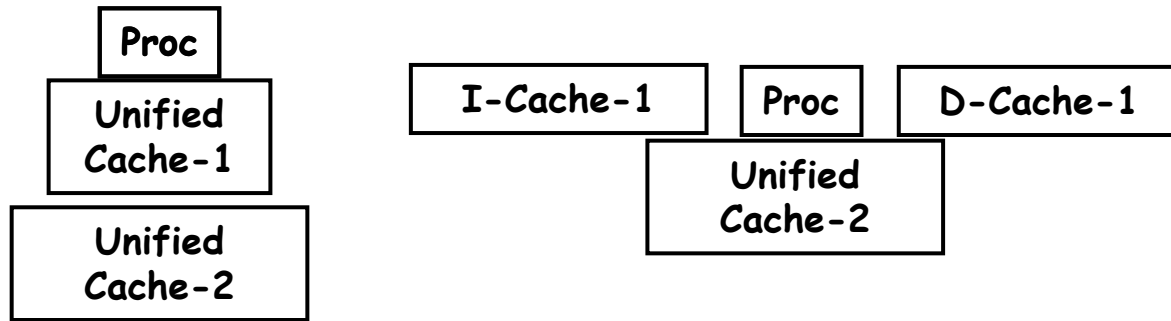
3Cs Absolute Miss Rate



Larger Block Size



Unified vs Split Caches



Example:

- 16KB I&D: Inst miss rate=0.64%, Data miss rate=6.47%
- 32KB unified: Aggregate miss rate=1.99%

◦ Which is better (ignore L2 cache)?

- Assume 33% data ops $\Rightarrow$ 75% accesses from instructions (1.0/1.33)
- hit time=1, miss time=50
- Note that *data* hit has 1 stall for unified cache (only one port)

$$AMAT_{\text{Harvard}} = 75\% \times (1 + 0.64\% \times 50) + 25\% \times (1 + 6.47\% \times 50) = 2.05$$

$$AMAT_{\text{Unified}} = 75\% \times (1 + 1.99\% \times 50) + 25\% \times (1 + 1 + 1.99\% \times 50) = 2.24$$

Review: Four Questions for Memory Hierarchy Designers

- Q1: Where can a block be placed in the upper level? (*Block placement*)
 - Fully Associative, Set Associative, Direct Mapped
- Q2: How is a block found if it is in the upper level? (*Block identification*)
 - Tag/Block
- Q3: Which block should be replaced on a miss? (*Block replacement*)
 - Random, LRU, direct map
- Q4: What happens on a write? (*Write strategy*)
 - Write Back or Write Through (with Write Buffer)

Review: Cache Organization

- *Direct mapped cache: n blocks per cache.*



Map block i from memory to $\text{mod } i/n$ of cache, where n is the number of blocks in cache
 $\Rightarrow$ Congruent mapping

- *Set-associative mapping: s sets, n/s blocks per set.*



Map block i from memory to set $\text{mod } i/n$ of cache, where n is the number of sets in cache.
Put the block into any block inside the set.

- *Fully associative mapping: one set per cache ($s = n$).*



Map block i from memory to any block of the cache

Review: Cache performance

- **Miss-oriented Approach to Memory Access:**

$$CPUtime = IC \times \left(CPI_{Execution} + \frac{MemAccess}{Inst} \times MissRate \times MissPenalty \right) \times CycleTime$$

$$CPUtime = IC \times \left(CPI_{Execution} + \frac{MemMisses}{Inst} \times MissPenalty \right) \times CycleTime$$

- $CPI_{Execution}$ includes ALU and Memory instructions

- **Separating out Memory component entirely**

- **AMAT = Average Memory Access Time**

- CPI_{ALUOps} does not include memory instructions

$$CPUtime = IC \times \left(\frac{AluOps}{Inst} \times CPI_{AluOps} + \frac{MemAccess}{Inst} \times AMAT \right) \times CycleTime$$

$$AMAT = HitTime + MissRate \times MissPenalty$$

$$= (HitTime_{Inst} + MissRate_{Inst} \times MissPenalty_{Inst}) + (HitTime_{Data} + MissRate_{Data} \times MissPenalty_{Data})$$

Book Ref

Topics:

1. **Virtual Machines** and Virtual Memory: Sections 5.6-5.7
2. Cache Memory: Sections 5.1-5.5

Assignment I Due on Sat 17 Apr 1500